Pre-trained models for generation tasks

NTV 2026 / HC6b

Today's lecturer: Sandro Pezzelle s.pezzelle@uva.nl

Table of contents

1. Updates & announcements
2. Recap
3. Transformers: Encoder-only
4. Encoder-decoder models
5. Decoder-only models
6. Wrap-up

Updates & announcements

Updates & announcements

- Very last lecture of the course!

Updates & announcements

- Very last lecture of the course!
- T6 due on Monday, March 23

Updates & announcements

- Very last lecture of the course!
- T6 due on Monday, March 23
- E6 available on ANS

Updates & announcements

- Very last lecture of the course!
- T6 due on Monday, March 23
- E6 available on ANS
- End-term exam next week > Practice exam on Canvas soon!

Updates & announcements

- Very last lecture of the course!
- T6 due on Monday, March 23
- E6 available on ANS
- End-term exam next week > Practice exam on Canvas soon!
- Please fill in the **course evaluation** when available! It will help us tune and improve the next editions.

Recap

From word embeddings to text encoders

Word embeddings:

- Learn word (less frequently, subword) embeddings, i.e., a good vector representation for 'dog', 'sun', 'freedom', 'eating', etc.

From word embeddings to text encoders

Word embeddings:

- Learn word (less frequently, subword) embeddings, i.e., a good vector representation for 'dog', 'sun', 'freedom', 'eating', etc.
- Can be learned via neural networks (*word2vec* [4], *fastText* [2]) or based on co-occurrence statistics

From word embeddings to text encoders

Word embeddings:

- Learn word (less frequently, subword) embeddings, i.e., a good vector representation for 'dog', 'sun', 'freedom', 'eating', etc.
- Can be learned via neural networks (*word2vec* [4], *fastText* [2]) or based on co-occurrence statistics (*GloVe* [5], seen in T3!)
- very good for semantic similarity tasks, retrieval, etc.

From word embeddings to text encoders

Word embeddings:

- Learn word (less frequently, subword) embeddings, i.e., a good vector representation for 'dog', 'sun', 'freedom', 'eating', etc.
- Can be learned via neural networks (*word2vec* [4], *fastText* [2]) or based on co-occurrence statistics (*GloVe* [5], seen in T3!)
- very good for semantic similarity tasks, retrieval, etc.
- such representations are said to be **static**: one vector representation per vocabulary entry, no matter the context

From word embeddings to text encoders

Word embeddings:

- Learn word (less frequently, subword) embeddings, i.e., a good vector representation for 'dog', 'sun', 'freedom', 'eating', etc.
- Can be learned via neural networks (*word2vec* [4], *fastText* [2]) or based on co-occurrence statistics (*GloVe* [5], seen in T3!)
- very good for semantic similarity tasks, retrieval, etc.
- such representations are said to be **static**: one vector representation per vocabulary entry, no matter the context
- some consequences: various word senses conflated into one representation (e.g., 'bank'); unawareness of surrounding words in a sentence; etc.

From word embeddings to text encoders

Text encoders:

- Give me a representation of an entire text: a sentence, a document, a book, etc.

From word embeddings to text encoders

Text encoders:

- Give me a representation of an entire text: a sentence, a document, a book, etc.
- Can be an **RNN** initialized with pre-trained word embeddings. Some problems:

From word embeddings to text encoders

Text encoders:

- Give me a representation of an entire text: a sentence, a document, a book, etc.
- Can be an **RNN** initialized with pre-trained word embeddings. Some problems:
  - starting from *static* word representations that need to be updated/fine-tuned
  - lack of parallelism/recurrent bottleneck (sequence is summarized—left to right—up to that point)
  - limited ability to capture long-range dependencies (multiple recurrent steps)
  - OOV/rare words poorly represented: reliance on limited vocabulary and unknown-token handling (UNK)

From word embeddings to text encoders

Text encoders:

- Give me a representation of an entire text: a sentence, a document, a book, etc.
- Can be a **pre-trained Transformer**! In particular, an encoder-only Transformer (e.g., BERT), jointly pre-trained on two objectives: masked language modeling (MLM) and NSP (next-sentence prediction). Some advantages:

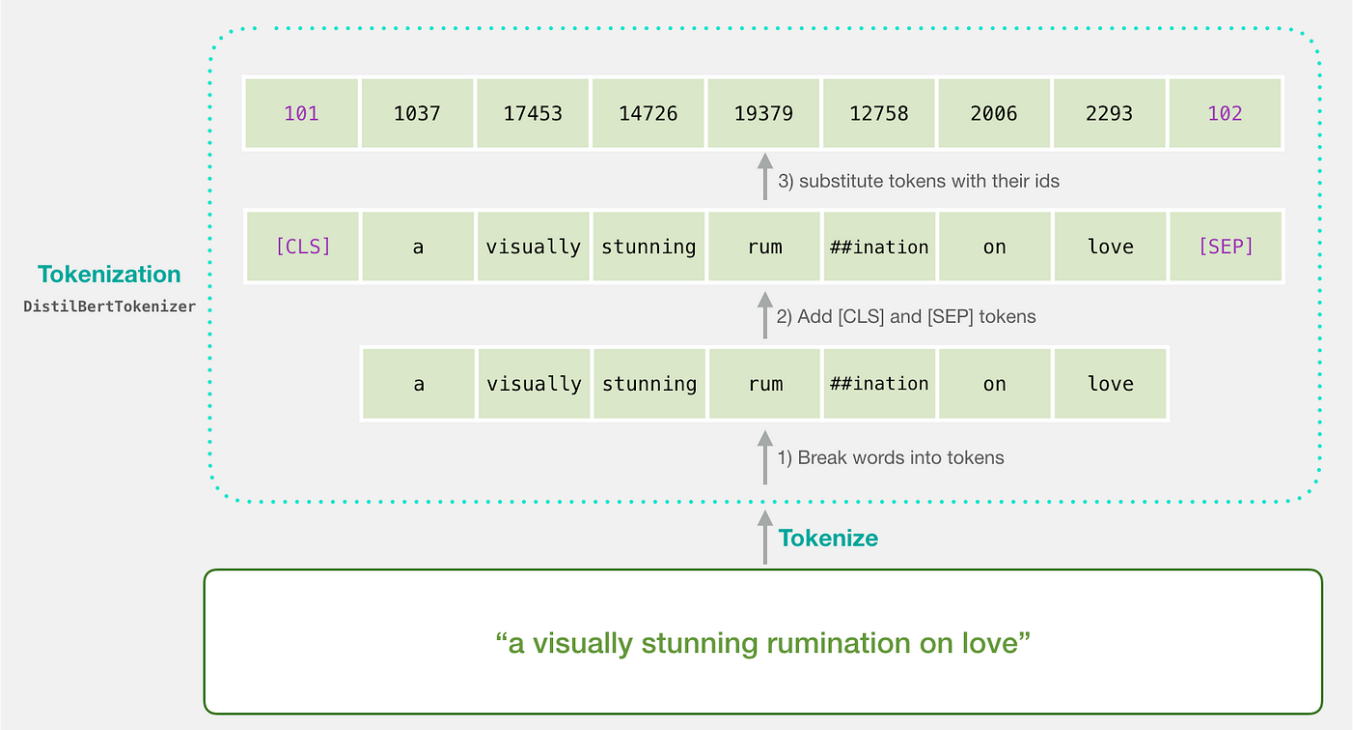
From word embeddings to text encoders

Text encoders:

- Give me a representation of an entire text: a sentence, a document, a book, etc.
- Can be a **pre-trained Transformer**! In particular, an encoder-only Transformer (e.g., BERT), jointly pre-trained on two objectives: masked language modeling (MLM) and NSP (next-sentence prediction). Some advantages:
  - learns contextualized embeddings that are aware of the surrounding context
  - parallelized solution/no recurrent bottleneck: at every layer, the model re-encodes the whole sequence (no summary)
  - improved ability to capture long-range dependencies (all input is always available)
  - works with **subword tokenizers** that allow for representing OOV/rare words by combining these representations

Subword tokenization

BERT's tokenizer is called ***WordPiece***: it uses its own vocabulary of subwords (not necessarily corresponding to meaningful linguistic units; found via engineering processes to obtain a manageable vocabulary of 30-50K entries)



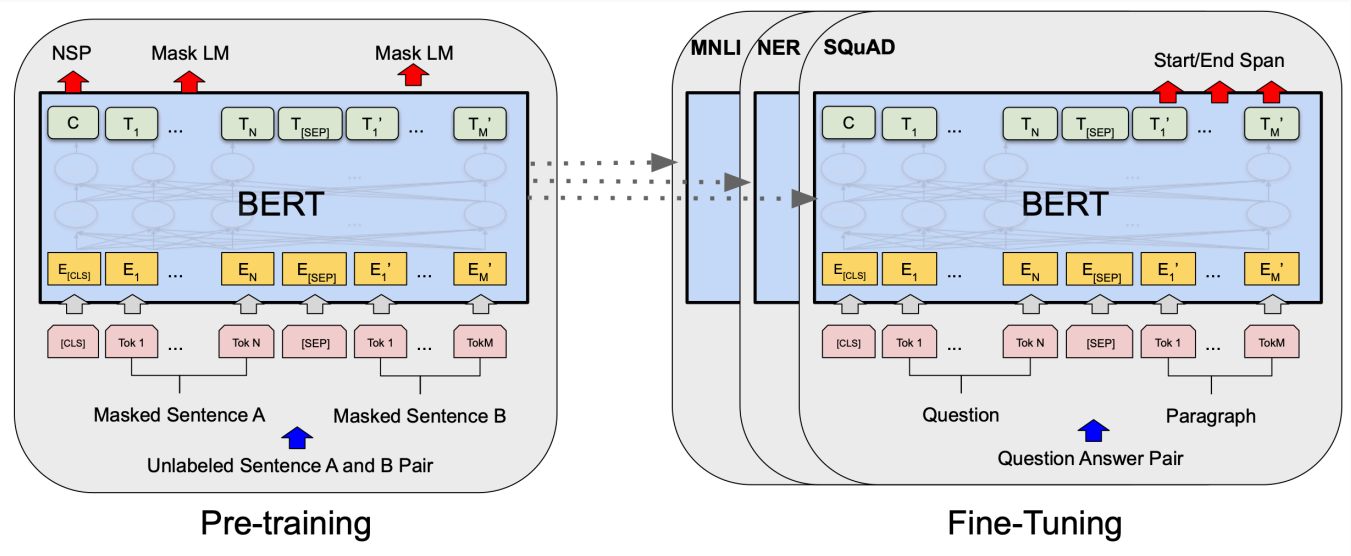
Text encoders & task-specific fine-tuning

Text encoders:

- Can be used as a backbone to solve many specific NLP tasks: text classification (one text mapped to a class), sequence labelling (one state mapped to one label), etc.
- To achieve so, we usually add a **classification/regression head** (one or more learnable layers) on top of the output of the text encoder—the [*CLS*] token for text classification, single states for sequence labelling, and so on.
- These heads are trained (trainable parameters); the encoder's weights can be fine-tuned or kept 'frozen', which reduces the computation needed.

Bidirectional Encoder Representations from Transformers

The BERT encoder [1] is a deep Transformer stack trained via MLM (as well as 'next sentence prediction') on Books Corpus (800m words) and English Wikipedia (2500m words).



BERT is pretrained on a combination of two objectives (MLM and NSP) on a large English corpus. After training, it can be adapted to a variety of tasks via, for example, fine-tuning the architecture and whatever necessary output layers on task-specific data and loss.

Transformers: Encoder-only

Encoder-only Transformers

Examples: BERT (in T6!), RoBERTa, DeBERTa, ELECTRA

Encoder-only Transformers

Examples: BERT (in T6!), RoBERTa, DeBERTa, ELECTRA

Key characteristics

- Stack of transformer encoder layers only
- Bidirectional self-attention (see both left and right context)
- Trained with masked language modelling and related pretraining tasks (e.g., NSP)
- Produce rich contextual representations for all tokens

Encoder-only Transformers

Examples: BERT (in T6!), RoBERTa, DeBERTa, ELECTRA

Key characteristics

- Stack of transformer encoder layers only
- Bidirectional self-attention (see both left and right context)
- Trained with masked language modelling and related pretraining tasks (e.g., NSP)
- Produce rich contextual representations for all tokens

Typical use cases

- Text classification (sentiment, topic)
- Sequence labelling (NER, PoS tagging, slot filling)
- Sentence / document embeddings (retrieval, semantic similarity)

Limitations of Encoder-only models

General limitations

- No built-in conditional generation mechanism:
  - Trained with MLM, not left-to-right next-token prediction
  - Poor fit for tasks requiring free-form output (see below)
- Difficult to handle input–output length mismatch:
  - Encoder-only gives one sequence of hidden states for the input
  - No way to iteratively generate a longer/shorter target sequence

Limitations of Encoder-only models

General limitations

- No built-in conditional generation mechanism:
  - Trained with MLM, not left-to-right next-token prediction
  - Poor fit for tasks requiring free-form output (see below)
- Difficult to handle input–output length mismatch:
  - Encoder-only gives one sequence of hidden states for the input
  - No way to iteratively generate a longer/shorter target sequence

Task-specific issues

- Machine translation / text summarization:
  - Need sequence-to-sequence mapping; encoder-only is mainly useful as a *feature extractor* for another decoder
- Question answering:
  - Span extraction is feasible (predict start/end indices), but
  - **open-ended, generative** QA is not naturally supported without adding a separate decoder

From classification to generation tasks

Examples:

- translation (e.g., NL to EN)
- summarization (from a long text to a summary/abstract)
- question answering (open-ended/free-form)
- dialogue (chatbots)
- also multimodal tasks: image captioning, visual question answering, visual dialogue ...

Classification tasks: input is text, output is a label
Generation tasks: input is text, output is also a text/also a sequence

Encoder-decoder models

Encoder–decoder Transformers

Examples: Original Transformer (Vaswani et al.), T5, BART, mBART

Key characteristics

- Two stacks: encoder and decoder % item Decoder: causal self-attention + cross-attention to encoder outputs
- Designed for **conditional sequence-to-sequence** generation, solving the one-to-one bottleneck of sequence labelling classification problems

Encoder–decoder Transformers

Examples: Original Transformer (Vaswani et al.), T5, BART, mBART

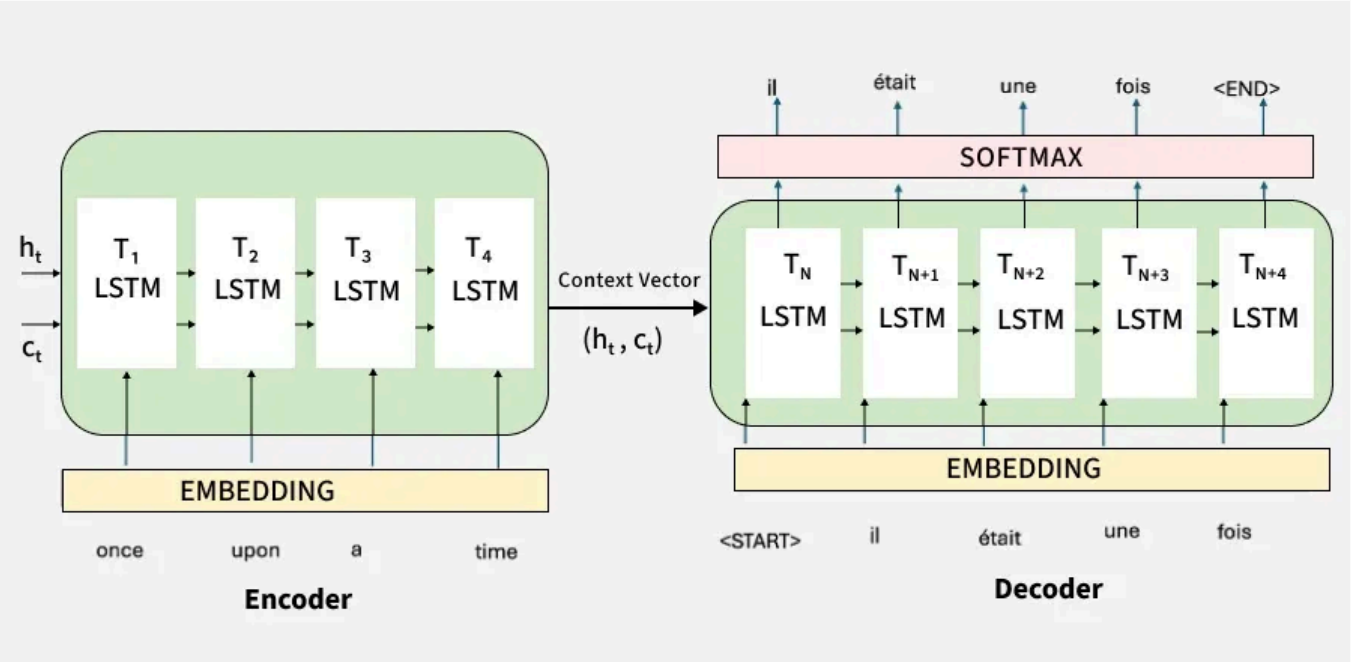
Key characteristics

- Two stacks: encoder and decoder % item Decoder: causal self-attention + cross-attention to encoder outputs
- Designed for **conditional sequence-to-sequence** generation, solving the one-to-one bottleneck of sequence labelling classification problems

Typical use cases

- Machine translation
- Text summarization
- Data-to-text / text-to-text generation
- Grammatical error correction, style transfer, paraphrasing

Encoder-decoder LSTMs

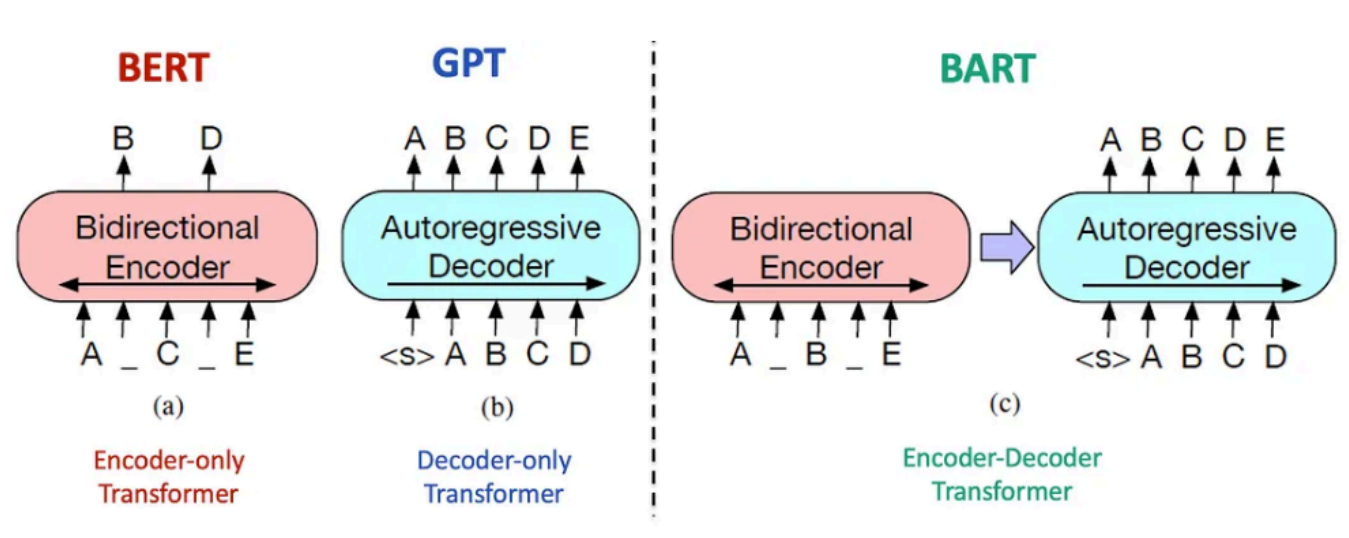


In an encoder-decoder architecture based on LSTMs, the encoder processes the source sequence and produces its final hidden state (and cell state). These are passed to the decoder as its initial state: This is the "context vector" that summarizes the source sentence.

$$\left(h_1^{\text{dec}}, c_1^{\text{dec}}\right) = \text{LSTM}\left(x_1^{\text{dec}},\ \left(h_t^{\text{enc}}, c_t^{\text{enc}}\right)\right)$$

Pre-trained Encoder-decoder

BART (Bidirectional and Auto-Regressive Transformers) [3], developed by Facebook (now Meta) in 2019, combines the strengths of BERT and GPT (Generative Pre-trained Transformer), models from Google and OpenAI, respectively.



BART

Overview

- BART is a **sequence-to-sequence** Transformer with:
  - A BERT-like encoder (bidirectional self-attention)
  - A GPT-like decoder (causal self-attention + cross-attention)

BART

Overview

- BART is a **sequence-to-sequence** Transformer with:
  - A BERT-like encoder (bidirectional self-attention)
  - A GPT-like decoder (causal self-attention + cross-attention)

Pre-training (denoising objective)

- Input text is corrupted by noise functions:
  - Token masking / deletion
  - Text infilling (replace spans with a single *<mask>*)
  - Sentence permutation / document shuffling
- The encoder receives the noisy text; the decoder is trained to reconstruct the original clean text token by token
- Objective: maximize likelihood of the original sequence given the corrupted one

BART

Overview

- BART is a **sequence-to-sequence** Transformer with:
  - A BERT-like encoder (bidirectional self-attention)
  - A GPT-like decoder (causal self-attention + cross-attention)

BART

Overview

- BART is a **sequence-to-sequence** Transformer with:
  - A BERT-like encoder (bidirectional self-attention)
  - A GPT-like decoder (causal self-attention + cross-attention)

Fine-tuning and inference

- At fine-tuning time, cast tasks into a **text-to-text** format:
  - E.g. summarization: *[ARTICLE]* $\rightarrow$ *[SUMMARY]*
  - Translation: source sentence $\rightarrow$ target sentence
  - QA: question + passage $\rightarrow$ answer
- **Inference**:
  - Encode the input text with the encoder
  - Use the decoder to **autoregressively** generate output tokens (e.g. via greedy decoding or beam search), attending to encoder states

Fine-tuning example

Fine-tuning for machine translation, ingredients:

- replace language corruption with a language change:
  - Input = source-language sentence (e.g., EN)
  - Output = target-language sentence (e.g., NL)
- **Train** with standard seq2seq cross-entropy (minimize the cross-entropy loss over all target positions)
- **Inference**: encode source, then autoregressively decode the target sentence

How does pre-training help?

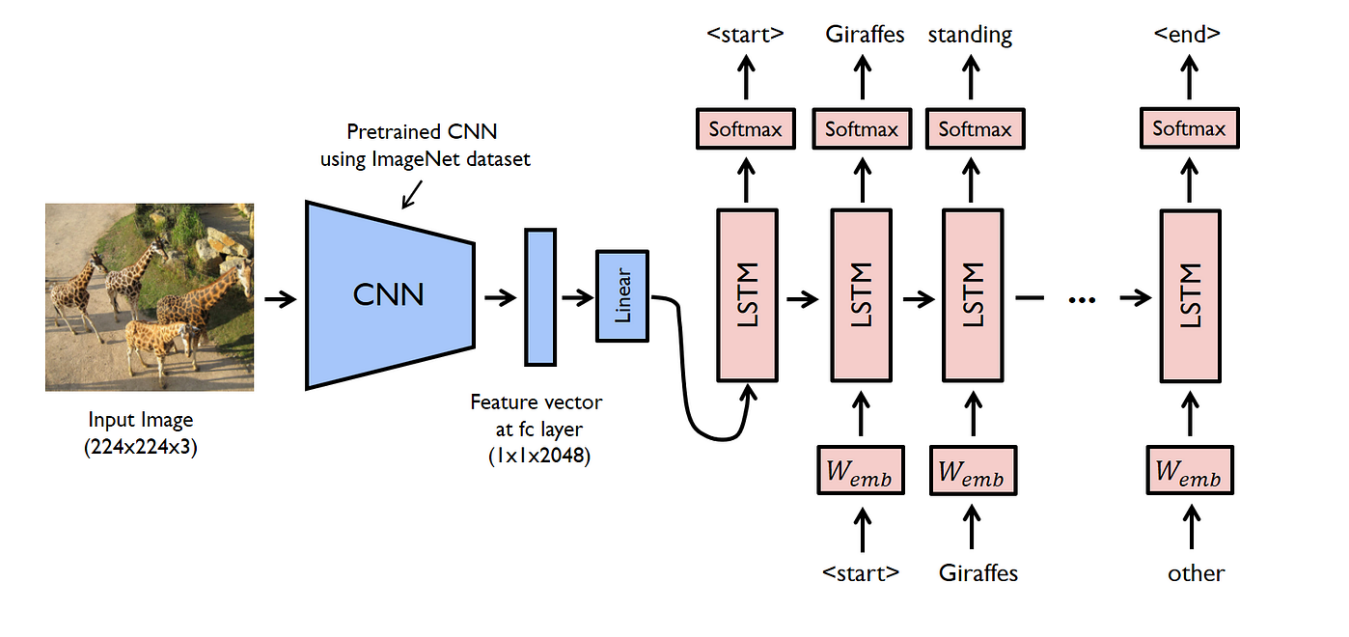
Pre-training teaches the model to:

- **Encode meaning robustly.** This is needed for translation: the model must ignore superficial form and focus on meaning
- **Generate fluent text conditioned on a source.** Translation is also about generating a fluent target sentence given some semantic representation (source sentence)

Cross-lingual mapping (source language → target language) is learned during fine-tuning on parallel data; pre-training just makes this much easier and more data-efficient!

Works also multimodally!

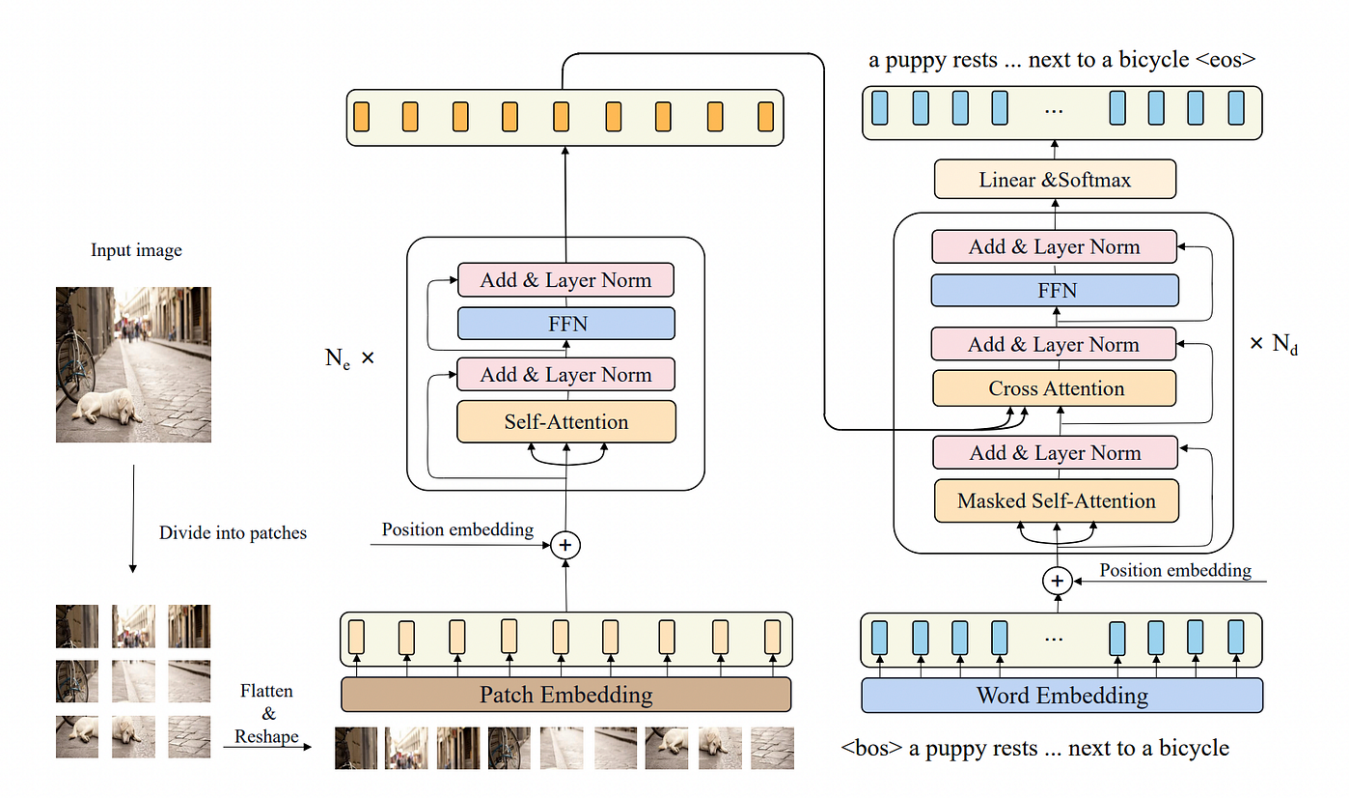
For example, for Image Captioning (IC):



With CNNs + LSTMs

Works also multimodally!

For example, for Image Captioning (IC):



With Transformers

Decoder-only models

Decoder-only Transformers

Examples: GPT family, LLaMA, PaLM, Mistral

Key characteristics

- Stack of transformer decoder layers only
- Causal (unidirectional) self-attention: each token attends only to past tokens
- Trained with **next-token prediction** on large text corpora
- Strong at autoregressive generation and in-context learning

Decoder-only Transformers

Examples: GPT family, LLaMA, PaLM, Mistral

Key characteristics

- Stack of transformer decoder layers only
- Causal (unidirectional) self-attention: each token attends only to past tokens
- Trained with **next-token prediction** on large text corpora
- Strong at autoregressive generation and in-context learning

Typical use cases

- Open-ended text generation (stories, code, dialogue)
- Chatbots / assistants and tool-using agents
- Few-shot / zero-shot classification via prompting
- Code completion and program synthesis

GPT Language Model Family

Pre-training data

- Very large, diverse corpora: web pages, books, articles, code, etc.
- Mixed domains and styles: dialogue, code, technical texts

Objective & loss

- **Next-token prediction** (autoregressive language modelling)
- Model learns $p_\theta(w_t \mid w_{<t})$ over sequences
- Loss: token-level **cross-entropy** summed/averaged over positions:

$$\mathcal{L} = -\sum_t \log p_\theta(w_t \mid w_{<t}).$$

Adaptation & fine-tuning

- **Instruction tuning** and often **RLHF**
- Few-shot / zero-shot via prompting (in-context learning)

Repurpose decoder-only models

Decoder-only models

- Input and output are treated as a *single sequence*:
  - Prompt: *`[instruction / source text / examples] + [answer starts here]`*
  - Model simply continues the sequence due to next-token prediction

Repurpose decoder-only models

Decoder-only models

- Input and output are treated as a *single sequence*:
  - Prompt: *`[instruction / source text / examples] + [answer starts here]`*
  - Model simply continues the sequence due to next-token prediction
- This unified interface turns many former *`seq2seq`* tasks into prompted generation:
  - Summarization: *``Summarize: [document]''*
  - Machine translation: *``Translate to German: [sentence]''*
  - QA: *``Question: ... Answer:''*
  - Style transfer, rewriting, code generation, etc.

Repurpose decoder-only models

Decoder-only models

- Input and output are treated as a *single sequence*:
  - Prompt: *`[instruction / source text / examples] + [answer starts here]`*
  - Model simply continues the sequence due to next-token prediction
- This unified interface turns many former *`seq2seq`* tasks into prompted generation:
  - Summarization: *``Summarize: [document]''*
  - Machine translation: *``Translate to German: [sentence]''*
  - QA: *``Question: ... Answer:''*
  - Style transfer, rewriting, code generation, etc.
- No separate encoder needed: the model **encodes** the input in its hidden states while reading the prompt and then **decodes** by continuing the text.

Wrap-up

The end

- Natural Language Processing is rapidly evolving and becoming more and more central in many AI applications (also multimodal)
- A lot of potential, but also risks we should be aware of (model size & footprint, data issues, reliance on human annotators, model closeness, access to technology, etc.)
- If you liked these topics: More NLP in the BKI and BSc AI programs
- Thank you for your participation and **success with the exam!**

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL *https://aclanthology.org/N19-1423/*.

[2] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomáš Mikolov. Bag of tricks for efficient text classification. In *Proceedings of the 15th conference of the European chapter of the association for computational linguistics: volume 2, short papers*, pages 427–431, 2017.

[3] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7871–7880, 2020.

[4] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[5] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.